

Improving Learning-based Camera Pose Estimation for Image-based Augmented Reality Applications

Enyu Cai*
cenyu@purdue.edu
Purdue University
West Lafayette, IN, USA

Ryan Rossi
ryrossi@adobe.com
Adobe Research
San Jose, CA, USA

Chang Xiao
cxiao@adobe.com
Adobe Research
San Jose, CA, USA

ABSTRACT

Camera tracking is essential for many augmented reality (AR) applications, such as rendering virtual content on top of a display. While marker-based camera tracking methods can accurately estimate the camera pose, they require pre-placed markers and occupy valuable screen space, potentially impacting the user experience. In contrast, markerless camera tracking methods do not have these limitations but tend to be less stable. In this work, we propose an improved approach for camera tracking that utilizes salient visual features commonly found on websites. We develop robust algorithms for detecting these features and design efficient methods for estimating the camera pose from them. Our approach outperforms state-of-the-art methods in terms of robustness, as demonstrated by our experiments. This work paves the way for a wide range of important AR applications, such as online shopping and interactive AR games.

CCS CONCEPTS

• **Human-centered computing** → *Ubiquitous and mobile computing systems and tools.*

KEYWORDS

Camera tracking, robustness

ACM Reference Format:

Enyu Cai, Ryan Rossi, and Chang Xiao. 2023. Improving Learning-based Camera Pose Estimation for Image-based Augmented Reality Applications. In *Extended Abstracts of the 2023 CHI Conference on Human Factors in Computing Systems (CHI EA '23)*, April 23–28, 2023, Hamburg, Germany. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3544549.3585756>

1 INTRODUCTION

Augmented Reality (AR) has become a key feature in many useful mobile applications, enhancing scenarios such as online shopping and AR gaming through the addition of virtual content on top of a reference image plane [13, 16, 21] as shown in Figure 1. To provide a seamless AR experience, it is essential to have a stable, efficient, and accurate camera pose estimation. There are two major

approaches for camera pose estimation in AR: marker-based tracking and markerless tracking. Marker-based tracking methods [6, 8] require visual markers arranged in a pre-known layout on a flat surface (e.g., displayed on a monitor or printed on paper). When the camera observes these markers, the camera pose can be estimated using the detected corner points of the markers. Those methods provide fast and stable solutions for camera pose tracking. However, the markers need to be placed in advance, which increases the user's overhead and takes up valuable space (especially when displayed on the screen).

Markerless methods do not require any pre-setup besides the camera. These methods utilize the natural visual features of captured images for estimating the camera poses. Most of the major hand-held AR applications are adopting markerless tracking. However, detecting natural features is significantly less stable, and therefore markerless methods are known to be less stable than marker-based tracking methods. In addition, these methods require repeated and time-consuming scans of the environment to obtain robust visual features which is another overhead for the user experience.

In this work, we develop a robust approach for camera pose estimation using content displayed on the digital screen, such as a website, as the reference plane. Our method consists of a deep learning-based image matcher and a matching refinement process. Specifically, our approach takes a pair of grayscale images as input and estimates an initial, coarse point-to-point match. These initial matches are then refined using matching information from previous frames in the sequence. The final camera pose is then derived using the Perspective-n-Point (PnP) [15] technique with RANSAC [7]. Our experiments demonstrate the effectiveness of our approach compared to state-of-the-art methods, achieving more accurate estimates of translation and rotation in both close-up and long-range settings. We also conduct ablation studies and provide results for the rendering stability across multiple frames.



Figure 1: Use case of our method.

*This work was done while the author was an intern at Adobe Research

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

CHI EA '23, April 23–28, 2023, Hamburg, Germany

© 2023 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-9422-2/23/04.

<https://doi.org/10.1145/3544549.3585756>

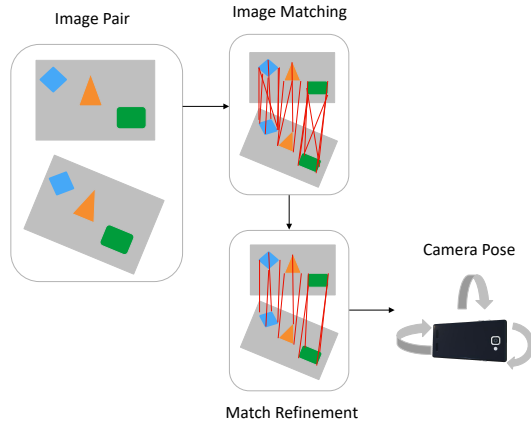


Figure 2: Pose estimation workflow of our method. The input image pair is going through the image matching and match refinement process to produce the camera pose output.

2 RELATED WORK

2.1 Camera Pose Estimation Methods

Marker-based camera pose estimation methods use fiducial markers to track the camera location. The fiducial markers usually contain a specific pattern with sharp edges and corners that can provide reference points on an image [6, 8]. The camera position is obtained by using the pre-defined measurement on the markers and the marker images captured by the camera, usually using ARToolkit [9]. For example, ARTag [6] is a fiducial marker that utilizes digital coding for an augmented reality application. Markerless methods for camera tracking can be split into structure-based methods and regression-based methods [22]. Structure-based methods use feature matching to find the correspondences between image and scene model [2, 3, 23]. The matching correspondence can be used by PnP to recover the camera pose. Regression-based methods use a single image as an input and output a camera pose directly via neural network [10, 11, 20].

2.2 Image Matching Methods

Image matching is the process of finding correspondences between features in a series of images. Image matching procedure often uses feature-based matching which involves extracting features from the images and comparing the features between images to find correspondences. For example, scale-invariant feature transform (SIFT) [14] is a feature-based method that uses scale-invariant feature detection with k-d tree algorithm. Speeded-up robust features (SURF) [1] is another feature-based algorithm that uses scale-space representation to detect feature points. In recent work, deep learning-based methods demonstrate capabilities on feature-based detection with high robustness such as D2-Net [5], SuperGlue [18] and LoFTR [19].

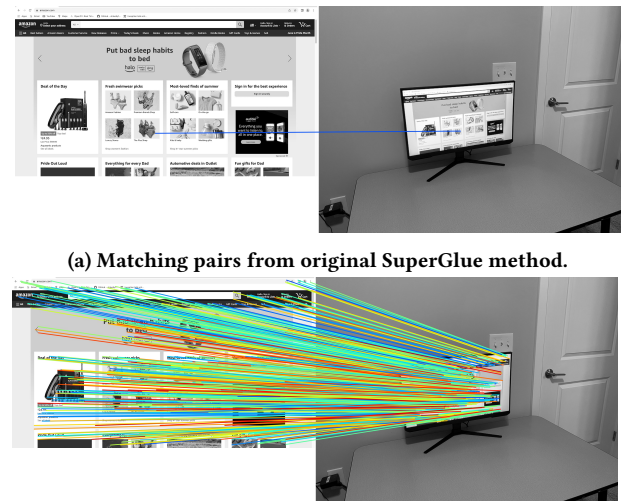
3 APPROACH

We designed an image-based approach for camera pose estimation as shown in Figure 2. The method consists of a deep learning-based image matcher and a match refinement procedure. The image matcher takes a pair of grayscale images as input and estimates

coarse point-to-point matches. These coarse matches are then refined using matching information from previous frames in the sequence by the refinement procedure. The final camera pose is generated by applying PnP pose computation with RANSAC to the refined matches. We retrained the deep learning-based image-matching method on a website dataset to improve its performance on the digital screen scenario. We also developed a refinement process to increase the accuracy of pose estimation.

3.1 Image Matching

The image matching consists of two parts: a feature point detector, and a matching approach that can receive the input from the feature detector. We use a deep learning-based SuperPoint [4] feature detector and descriptor. SuperPoint method is a fully convolutional neural network that can output SIFT-like point location and descriptor. Compared to SIFT, it has a faster inference time and can produce more feature points for matching. For image matching, we use SuperGlue [18], a deep learning-based method as the matcher. The matching results are further improved by our refinement algorithm. Compared to traditional image matching methods such as SIFT [14] and ORB [17], deep learning-based methods have better run-time and higher accuracy in the general dataset. Both methods are learning-based, so we can fine-tune them to achieve high accuracy by training on a dataset that has a similar scene to our application. We retrained the SuperGlue method on the Webis-WebSeg-20 dataset [12], which contains web page screenshot images. As we will discuss in Figure 3, fine-tuning increases the matching performance drastically.



(a) Matching pairs from original SuperGlue method.

(b) Matching pairs from SuperGlue trained on our webpage dataset.

3.2 Refinements

Although the matching results from the SuperGlue method have acceptable accuracy for a single frame, they are not accurate enough for rendering AR content on a display due to the instability of matches across multiple frames. To improve the accuracy of pose

estimation, we refine the matches from SuperGlue by selecting the region of interest for feature detection and removing unstable matches. The original SuperPoint method is highly sensitive to background information and can produce a large number of feature points in the background, diluting the number of points in the area of interest and affecting the quality of pose estimation. To eliminate feature points detected in the background, we extract the monitor screen from the surroundings so that all feature points are located in the region of interest. We use the matching information from the previous frame to build a mask for screen segmentation, which is padded with additional pixels to compensate for movement between frames. This approach uses existing information from matches and has a minimal impact on runtime.

Algorithm 1: Stable matching filtering

Input : a threshold δ , a $m \times n$ matrix S contains m matches across n frames, a $1 \times m$ zero matrix c stores counting for every matches

Output: a $1 \times N$ stable matching mapping c

```

for  $j = 1$  to  $n$  do // Iterate through frames
     $c_j \leftarrow c_j + S_{*,j}$  // Add match count to  $c$ ,  $S_{*,j}$  is
        the match occurrence for  $j$  frame
end
 $c[c \leq \delta] \leftarrow 0$  // Set the mapping vector to preserve
    stable matches
return  $c$ 

```

Some of the matches from SuperGlue are not stable due to the blurring frames caused by the movement and vibration of the camera. The unstable matching often shows up and disappears in consecutive frames which significantly decreases the pose estimation accuracy. To address this issue, our approach discovers high-quality matches that are highly stable. In order to find such highly stable matches, we consider the matches in frames from the first few seconds and record them. We define the stable matches if the total number of presents across multiple frames are greater than a certain threshold δ . Let S denote a matrix that encodes whether a match i is presented in a frame j or not. More formally,

$$S_{i,j} = \begin{cases} 1 & \text{if match exists} \\ 0 & \text{else} \end{cases} \quad (1)$$

Further, we also define a mapping vector c that defines the stability for a match i :

$$c_i = \begin{cases} 1 & \text{if stable match} \\ 0 & \text{else} \end{cases} \quad (2)$$

We can use S and c to extract the stable matches for a frame. The algorithm 1 describes the method to find stable matches. The match occurrence for the first n frames in matrix S is summed up to obtain c to count the total number present for every match. The unstable match is filtered out by δ which will set the mapping on that specific match to 0 if the number of occurrences is less than δ . The output is a stable match mapping c which can be used to extract the stable matches in the next frames. Assume a matrix M contains matching information of a frame for rendering, the stable matches M_{stable} can be extracted by:

$$M_{stable} = M \wedge c \quad (3)$$

where $\wedge$ is the logical "and" operator.

3.3 Leveraging Stable Matches

As mentioned before, we want to use stable matches to avoid the instability of continuous pose estimation. The stable matches M_{stable} are used to solve PnP pose computation for pose estimation. For this, we want the matches to be as stable as possible so the threshold δ is set to the number of frames (n) for the first few seconds. Notice that if we increase the time too much to find stable matches, it will dramatically affect the total number of available matches for pose estimation since not every match is guaranteed to have 100% occurrences in every frame. We set the time to the first 4 seconds (240 frames) for finding stable matches empirically since it is not too short to find stable matches and not too long to affect the number of available matches.

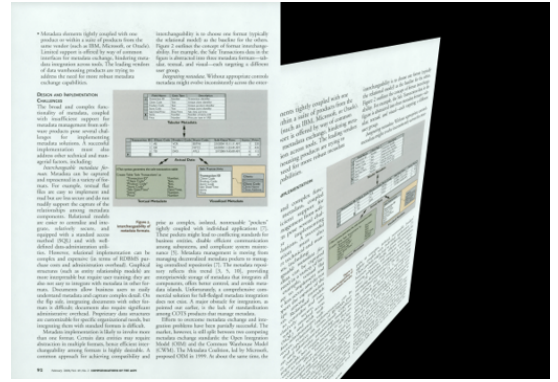


Figure 4: Random homography example. Left is the original image and right is the image with random homography applied.

3.4 Dataset

We use Webis-WebSeg-20 [12] dataset that contains 8,490 web pages for fine-tuning the SuperGlue model. Random homographies are applied to each image in order to produce the ground truth. Figure 4 shows an example of random homography applied to one of the images in the dataset. Each image is cropped and resized to 480x640 for training purposes. We split 80% images for training and 20% for validation. We use 8 NVIDIA A100 40G GPU for training and the total training time is around 18 hours.



Figure 5: Dataset collection setup. Images are extracted from the videos recorded by phone.

4 EXPERIMENTS

4.1 Experimental Setup

We compared our methods with the traditional image-matching method, SIFT, and the markerless-based method, Apple ARKit. We developed our own evaluation setup because there is no public benchmark that is tied specifically to our application. We built a dataset that contains 15 different camera directions of 4 web content images on a monitor screen. We chose to use images from the monitor screen rather than printed material because it is more challenging for image-based methods and allows us to generate accurate ground truth for comparison. The color of the content viewed on the monitor will slightly change depending on the viewing angle and can create artificial patterns if the camera is moved too close to the monitor, both of which can affect the accuracy of image-based methods for pose estimation. We used a marker-based method (checkerboard) as our ground truth for camera location, and the use of a monitor allowed us to switch between content and checkerboard patterns in the same region, helping to produce accurate results for ground truth. The setup is shown in Figure 5. We use a 27-inch 1080p monitor as the main display. We use an iPhone 13 Pro Max smartphone on a tripod to record a video of a monitor screen. The recorded video has 1920x1440 resolution and 60fps. The video is captured by CamTrackAR APP that uses ARKit to generate the camera pose in a separate file. For each camera direction, the checkerboard pattern is present at first to set the ground truth. Then the webpage contents will replace the checkerboard image as the data used for image-based matching methods. After displaying the webpage contents, the monitor will display the checkerboard image again and the phone will move to the next position to repeat the collection process. To compare with the markerless method, we extract the individual frames that are associated with the checkerboard and web contents from the videos. In this way, we can collect checkerboard patterns, webpage images and ARKit pose information in a fixed position at the same time to have a consistent camera pose. For ARKit method, the floor surface is set to the table under the monitor. For SIFT and SuperGlue method, the matching is acquired by PnP pose computation with RANSAC. We uses two different distance setup, 0.5m and 1m that represents the distance between the phone and the monitor to examine the effect of distance on our methods.

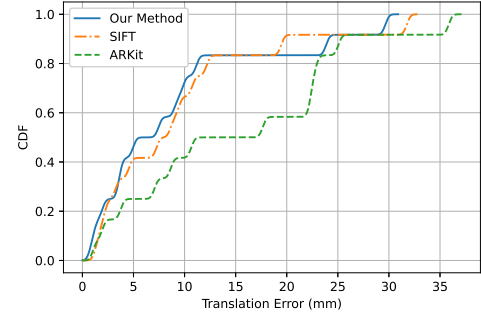
4.2 Metrics

We use translation error and rotation error as the main metrics for evaluation:

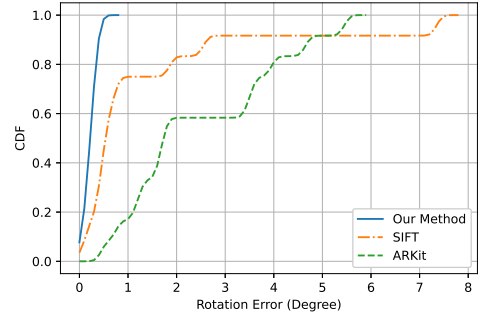
$$\text{Translation Error} = s(\|t_{vect_{gt}} - \|t_{vect_m}\|) \quad (4)$$

$$\text{Rotation Error} = \arccos(\text{trace}(R_{gt}^T R_m) - 1) / 2 \quad (5)$$

where $t_{vect_{gt}}$ and t_{vect_m} are the translation vector of ground truth and measurement. R_{gt} and R_m are the rotation matrix for ground truth and measurement. Further, s is the scale value that converts the translation unit to the real-world physical number (mm). We use the difference in movement between two camera pose to calculate t_{vect_m} and $t_{vect_{gt}}$. The scale of t_{vect_m} and $t_{vect_{gt}}$ are different so all the t_{vect_m} for each method are scaled to $t_{vect_{gt}}$ with average scale values. SIFT is running on an Intel Xeon 8275c1 CPU,



(a) 3D Translation Error 0 - 10mm.



(b) 3D Rotation Error 0 - 2°.

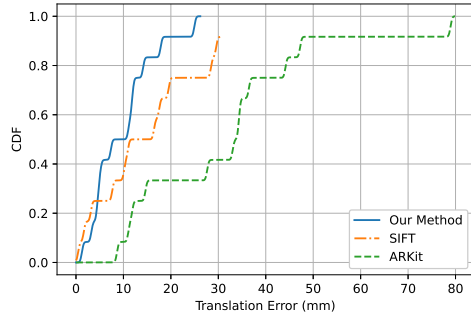
Figure 6: Results for Short Range Error. 3D camera pose estimation error in 0.5m compared with SIFT and ARKit.

our method is inferred on an NVIDIA A10g GPU and ARKit is running on iPhone 13 Pro Max.

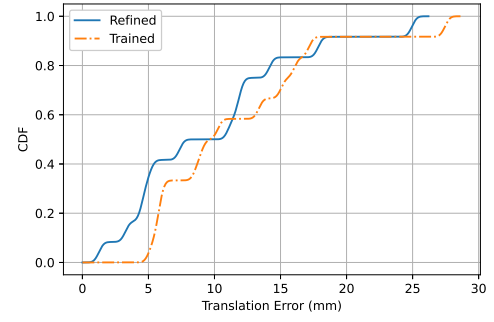
4.3 Evaluation Results on Still Images

We evaluate three methods on the images with 0.5m distance between the camera and the monitor screen. Figure 6 shows the cumulative distribution function (CDF) of translation error and rotation error. Notably, the median translation and rotation error of our approach is 6.2mm and 0.2°. In comparison, the median error of SIFT and ARKit is 22.9mm, 2.2° and 14.2mm, 1.7°, respectively.

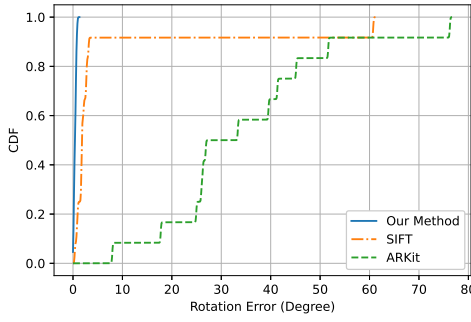
Our method outperforms SIFT and ARKit in both translation and rotation error. Nevertheless, our approach is significantly faster than SIFT. For instance, SIFT takes 1200ms per frame whereas our method takes 80ms. ARKit is the fastest method with an inference time of around 17ms. We also evaluated our method on the long-range images where the gap between the phone and monitor is 1m. Results are provided in Figure 7 where we show the CDF of translation error and rotation error. The translation and rotation median error of our method is 9.1mm and 0.4°, while the error for SIFT and ARKit is 13.7mm, 1.8° and 33.7mm, 30°, respectively. From the results, our method outperforms SIFT and ARKit. For the long-range setting, ARKit performs significantly worse while our approach is robust, remaining highly accurate in terms of 3D translation and rotation as shown in Figure 7.



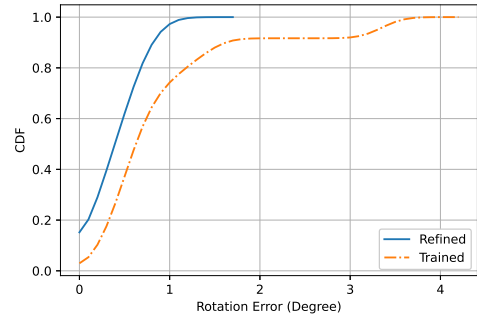
(a) 3D Translation Error 0 - 80mm.



(a) 3D Translation Error 0 - 30mm.



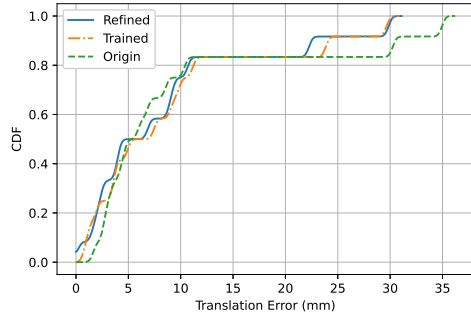
(b) 3D Rotation Error 0 - 80°.



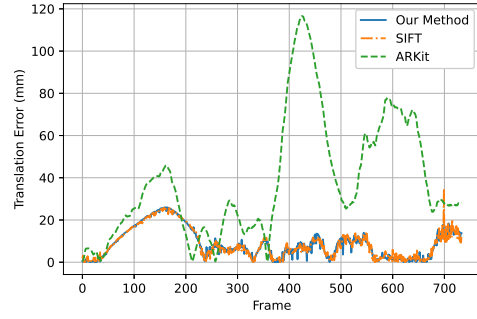
(b) 3D Rotation Error 0 - 4°.

Figure 7: Results for Long Range Error. 3D camera pose estimation error in 1m compared with SIFT and ARKit.

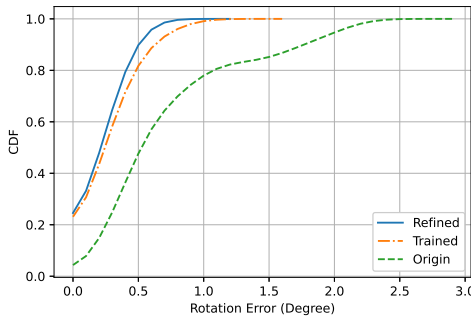
Figure 9: Ablation study results (1m). Note the original indoor SuperGlue cannot produce valid matches for pose estimation.



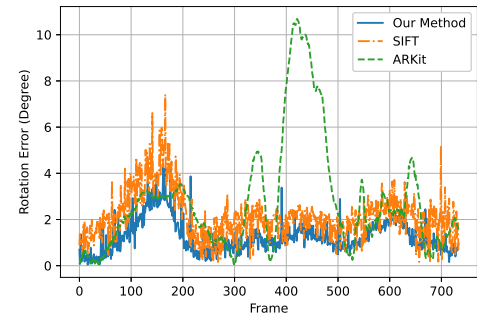
(a) 3D Translation Error 0 - 35mm.



(a) Translation Error for continuous frames.



(b) 3D Rotation Error 0 - 3°.



(b) Rotation Error for continuous frames.

Figure 8: Ablation study results (0.5m).

Figure 10: Stability evaluation from videos.

4.4 Ablation Study

We compared our refined approach, fine-tuned weight of SuperGlue, and the original weight of SuperGlue on the short-range and long-range dataset. The results are shown in Figure 8 and 9. For the short-range setting, the median error of the refinement process is 5.4mm, 0.2°, while the error of fine-tuned and origin weight is 6.2mm, 0.2° and 5.3mm, 0.5° respectively. For long-range error testing, we found the weight from the original SuperGlue method cannot produce enough matches for pose estimation. The median error of refinement is 9.1mm and 0.3°, while the error of the fine-tuned weight is 9.7mm, 0.6°, respectively. Compared to the original method, retraining the network with matching refinement can improve the performance on pose estimation, especially when the distance between the camera and object is far away from each other.

4.5 Rendering Stability Comparison on Video

To understand the stability of our approach, we compared the rendering between our method and ARKit using a box shape object rendered on the monitor screen. The ARKit rendering is made by Adobe Aero with a built-in image anchor function to fix the box location on the monitor. From the videos, the box shape from our method is much more stable than the ARKit results. We also evaluated the stability of a video with camera movements. The content and checkerboard pattern are placed side by side in order to acquire the ground truth information for every frame. The distance between the screen and the camera is 0.5m. The results are provided in Figure 10. Notably, we show the translation and rotation error for each approach across all the frames. From the video stability evaluation, our method performs significantly better than ARKit as shown in Figure 10. Notably, the average error of our method is 8.7mm, 1.2°, while the average error of ARKit and SIFT are 38.5mm, 2.8° and 8.7mm, 2.1°, respectively.

5 CONCLUSION

In this paper, we introduce a robust image-based camera tracking method using contents on digital screen. Our approach estimates the camera pose by using a deep learning based image matcher with match refinement process. Experimental results show our method can achieve better accuracy for camera tracking compare to current state-of-the-art markerless camera tracking method. This work paves the way for a wide range of AR applications, such as online shopping and interactive AR games. Future work will include optimize the running time of the workflow.

REFERENCES

- [1] Herbert Bay, Tinne Tuytelaars, and Luc Van Gool. 2006. SURF: Speeded Up Robust Features. In *Computer Vision – ECCV 2006*, Aleš Leonardis, Horst Bischof, and Axel Pinz (Eds.). Springer, Berlin, Heidelberg, 404–417. https://doi.org/10.1007/11744023_32
- [2] Eric Brachmann and Carsten Rother. 2018. Learning Less is More - 6D Camera Localization via 3D Surface Regression. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 4654–4662. <https://doi.org/10.1109/CVPR.2018.00489>
- [3] Eric Brachmann and Carsten Rother. 2022. Visual Camera Re-Localization From RGB and RGB-D Images Using DSAC. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44, 9 (2022), 5847–5865. <https://doi.org/10.1109/TPAMI.2021.3070754>
- [4] Daniel DeTone, Tomasz Malisiewicz, and Andrew Rabinovich. 2018. SuperPoint: Self-Supervised Interest Point Detection and Description. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. 337–33712. <https://doi.org/10.1109/CVPRW.2018.00060>
- [5] Mihai Dusmanu, Ignacio Rocco, Tomas Pajdla, Marc Pollefeys, Josef Sivic, Akihiko Torii, and Torsten Sattler. 2019. D2-Net: A Trainable CNN for Joint Description and Detection of Local Features. In *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 8084–8093. <https://doi.org/10.1109/CVPR.2019.00828>
- [6] Mark Fiala. 2005. ARTag, a fiducial marker system using digital techniques. In *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)*, Vol. 2. 590–596 vol. 2. <https://doi.org/10.1109/CVPR.2005.74>
- [7] Martin A. Fischler and Robert C. Bolles. 1981. Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography. *Commun. ACM* 24, 6 (jun 1981), 381–395. <https://doi.org/10.1145/358669.358692>
- [8] Michail Kalaitzakis, Sabrina Carroll, Anand Ambrosi, Camden Whitehead, and Nikolaos Vitzilaos. 2020. Experimental Comparison of Fiducial Markers for Pose Estimation. In *2020 International Conference on Unmanned Aircraft Systems (ICUAS)*. 781–789. <https://doi.org/10.1109/ICUAS48674.2020.9213977>
- [9] Hirokazu Kato and Mark Billinghurst. 1999. Marker tracking and HMD calibration for a video-based augmented reality conferencing system. In *Proceedings 2nd IEEE and ACM International Workshop on Augmented Reality (IWAR'99)*. 85–94. <https://doi.org/10.1109/IWAR.1999.803809>
- [10] Alex Kendall and Roberto Cipolla. 2016. Modelling uncertainty in deep learning for camera relocalization. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*. 4762–4769. <https://doi.org/10.1109/ICRA.2016.7487679>
- [11] Alex Kendall, Matthew Grimes, and Roberto Cipolla. 2015. PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization. In *2015 IEEE International Conference on Computer Vision (ICCV)*. 2938–2946. <https://doi.org/10.1109/ICCV.2015.336>
- [12] Johannes Kiesel, Lars Meyer, Florian Kneist, Benno Stein, and Martin Potthast. 2021. An Empirical Comparison of Web Page Segmentation Algorithms. In *Advances in Information Retrieval. 43rd European Conference on IR Research (ECIR 2021) (Lecture Notes in Computer Science, Vol. 12657)*. Springer, 62–74. https://doi.org/10.1007/978-3-030-72240-1_5
- [13] Changmin Lim, Chanran Kim, Jong-II Park, and Hanhoon Park. 2016. Mobile Augmented Reality Based on Invisible Marker. In *2016 IEEE International Symposium on Mixed and Augmented Reality (ISMAR-Adjunct)*. 78–81. <https://doi.org/10.1109/ISMAR-Adjunct.2016.0045>
- [14] David G. Lowe. 2004. Distinctive Image Features from Scale-Invariant Keypoints. *International Journal of Computer Vision* 60 (2004), 91–110. <https://doi.org/10.1023/B:VISI.0000029664.99615.94>
- [15] Eric Marchand, Hideaki Uchiyama, and Fabien Spindler. 2016. Pose Estimation for Augmented Reality: A Hands-On Survey. *IEEE Transactions on Visualization and Computer Graphics* 22, 12 (2016), 2633–2651. <https://doi.org/10.1109/TVCG.2015.2513408>
- [16] Hanhoon Park and Jong-Il Park. 2010. Invisible Marker-Based Augmented Reality. *International Journal of Human-Computer Interaction* 26, 9 (2010), 829–848. <https://doi.org/10.1080/10447318.2010.496335>
- [17] Ethan Rublee, Vincent Rabaud, Kurt Konolige, and Gary Bradski. 2011. ORB: An efficient alternative to SIFT or SURF. In *2011 International Conference on Computer Vision*. 2564–2571. <https://doi.org/10.1109/ICCV.2011.6126544>
- [18] Paul-Edouard Sarlin, Daniel DeTone, Tomasz Malisiewicz, and Andrew Rabinovich. 2020. SuperGlue: Learning Feature Matching With Graph Neural Networks. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 4937–4946. <https://doi.org/10.1109/CVPR42600.2020.00499>
- [19] Jiaming Sun, Zehong Shen, Yuang Wang, Hujun Bao, and Xiaowei Zhou. 2021. LoFTR: Detector-Free Local Feature Matching with Transformers. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 8918–8927. <https://doi.org/10.1109/CVPR46437.2021.00881>
- [20] Abhinav Valada, Noha Radwan, and Wolfram Burgard. 2018. Deep Auxiliary Learning for Visual Localization and Odometry. In *2018 IEEE International Conference on Robotics and Automation (ICRA)* (Brisbane, Australia). IEEE Press, 6939–6946. <https://doi.org/10.1109/ICRA.2018.8462979>
- [21] Chang Xiao, Ryan Rossi, and Eunye Koh. 2022. IMarker: Instant and True-to-Scale AR with Invisible Markers. In *Adjunct Proceedings of the 35th Annual ACM Symposium on User Interface Software and Technology*. Association for Computing Machinery, New York, NY, USA, Article 32, 3 pages. <https://doi.org/10.1145/3526114.3558721>
- [22] Meng Xu, Youchen Wang, Bin Xu, Jun Zhang, Jian Ren, Stefan Poslad, and Pengfei Xu. 2022. A Critical Analysis of Image-based Camera Pose Estimation Techniques. (2022). <https://doi.org/10.48550/ARXIV.2201.05816>
- [23] Luwei Yang, Ziqian Bai, Chengzhou Tang, Honghua Li, Yasutaka Furukawa, and Ping Tan. 2019. SANet: Scene Agnostic Network for Camera Localization. In *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*. 42–51. <https://doi.org/10.1109/ICCV.2019.00013>